

CODE SPORT



This month, we continue our discussion on testing machine learning applications, focusing on natural language processing (NLP).

In the last column, we started discussing why Natural Language Processing (NLP) models make glaring mistakes on a simple example, when apparently achieving state-of-art performance on their test data sets. When a NLP model shows poor performance on a trivial example, which humans can classify correctly, the immediate reactions seem to be one of the following:

1. We should just add more training data.
2. We should add more layers to the model and make it deeper.
3. We should take the latest pretrained model and fine tune it on our data.

None of these responses adequately address the problem. The first response presumes that the model is probably over-fitting. If that was the case, it would have shown up on the test set evaluation metrics. The second response appears reasonable at first glance, based on the rationale that the model is too simple to understand the complex relation encoded in the incorrectly predicted example. But this is not correct, since the example for which the model fails, is simpler than the ones the model already predicts correctly. This is not an under-fitting issue, per se. Hence building a more complex model by adding more layers or using a different neural unit is not likely to solve the problem. The third response is based on a fallacious assumption similar to that of the first response. Since a pretrained model is typically trained on a large real-world data corpus, it may be able to mitigate any data set's specific biases that the model may learn from your task's specific corpus. While this assumption appears reasonable, this does not often hold good in the real world. Data set-specific biases often get carried over even with fine-tuned pretrained models. What then can be done when dealing with this problem?

This problem happens because the model has been trained on an objective that is proxy for the original task that we wanted to solve. Let us clarify this with a quick example. Let us assume that our classifier is a paraphrase detection system.

Let us consider the following three examples:

- S1: Show me the flights from Bengaluru to Mumbai.
- S2: Show me the flights to Mumbai from BLR.
- S3: Show me the flights from Mumbai to Bengaluru.

It needs to classify S1 and S2 as paraphrases, whereas S1 and S3 are not paraphrases. However, it classifies both (S1, S2) and (S1, S3) as paraphrases. S1 and S3 have a high lexical overlap. All the words present in S1 are present in S3, though the word order is different. Let us now look at the data set on which you trained your classifier. If all the positive samples for paraphrase detection had a high degree of lexical overlap, it is possible that your classifier had learnt a shortcut heuristic to the task, based on the lexical overlap degree between the two sentences, instead of learning whether the two sentences are equivalent in meaning.

The classifier had no knowledge of the actual task it was supposed to learn. It needed to learn a function which could separate the positive samples from the negative ones in your training data. Given that most of the positive examples in the training data had a high degree of lexical overlap, the classifier learnt the shortcut of using lexical overlap as the means of separating the positive and negative examples. Hence, one way of solving this problem is to add positive paraphrase examples without lexical overlap and negative paraphrase examples that have a high degree of lexical overlap. This will force the classifier to learn a function that does not use the lexical overlap shortcut.